Chainy Game Design Document

Implementation Format

Gameplay First Edition

Version 2.0 • Adopted October 23, 2026 • Canonical game design

An offline island-restoration adventure for learning Web3, Blockchain and AI.

The player helps residents restore island services, uses a fictional digital economy, repairs a damaged local blockchain record, and teaches and checks an AI helper. Learning happens through consequential actions; short quizzes and vocabulary activities support those actions.

November 10, 2026 - Cryptita Plays community beta testing: deliver one complete Web3 restoration quest and one Blockchain repair challenge with dependable local saving. Add one AI training-and-test activity only after these pass Android checks. A broader three-world introduction is a stretch target, not a promise of 50 percent completion.

The owner reports that movement and dialogue are currently playable. The lead developer reports availability of up to 16-18 hours per day from October 23 through November 21. Neither statement is an independently tested build result. Estimates in Section 7 use focused implementation hours, not all available waking hours.

Adopted by the project owner on October 23, 2026, including the standalone Wordle and BlockMatch deferrals. This is the canonical GDD. Sherwin owns design decisions, Christian technical validation, Sean authored project documentation, Luis production art and Miguel UI implementation. Adoption specifies intended behavior; it does not establish implemented or tested behavior.

The beta is **Cryptita Plays community beta testing on November 10, 2026**; the former Marinduque school event is superseded. **Pre-final remains November 21, 2026.** Community testing must not be described as approved school/pilot-area research without the necessary confirmation. See the baseline, acceptance checklist, task plan, and GDD directory and change log. Earlier sources are preserved in the adoption archive.

All illustrated game screens are design concepts, not screenshots of a playable build. Generated artwork is not an import-ready Unity asset kit.

Contents

1 Game objectives and brief 3

2 Gameplay 4

3 Game world 5

4 Graphics and art style 10

5 Mechanics 13

6 Characters 19

7 Project plan and November 10 feasibility 20

8 References and template adaptation 26

9 Bibliography 28

This edition follows the supplied *Games Design Document - Project Vanishing* as its main structural reference: brief, gameplay and controls, world and locations, art and GUI development, storyboard and sample narrative, sound, systems, characters, production plan and sources. Chainy's own learning requirements replace that example's combat, weapon progression, reputation and PC controls.

Reading the scope labels

CORE means the adopted November 10 delivery boundary, subject to the first Android checkpoint. **STRETCH** means work admitted only after the core passes. **FULL** means retained in the full experience but not promised for November 10. **LATER SERVICE** means teacher provisioning, collection or distribution work deferred behind gameplay.

These labels are delivery boundaries, not evidence that a feature is complete. The school's completion formula is unknown. No task count, illustration, estimated hour or chapter count establishes its required percentage.

Main changes from the previous proposal

- Keep the three-world journey, named guides and twelve hands-on concepts.
- Put the playable restoration loop ahead of the teacher-service platform.
- Keep local saving in the core: it is essential gameplay, not optional analytics work.
- Replace a broad pre-beta wish list with explicit gates, recovery rules and fallback scope.
- Preserve the full AI idea, but sequence the small example-based classifier before the assistant and agent exercises.
- Separate the Blockchain repair exercise from the existing BlockMatch mini-game. They are not interchangeable names or implementations.

1 Game objectives and brief

1.1 Game brief

Chainy is a single-player, offline Unity/C# Android learning adventure for learners aged 12-17. It uses a fixed orthographic isometric view, compact low-poly environments, touchscreen movement and contextual interaction. English is the first language. No real cryptocurrency, real wallet, public blockchain, financial account or cloud AI service is required.

The intended player experience is curiosity followed by useful action: “I need to restore the workshop; I can earn a reward, buy a tool, check the ownership record and use the tool.” Definitions explain an action the learner has just encountered instead of becoming the whole activity.

One personal phone per learner is the intended school model. Once enrolled in the full product, play and saving do not depend on the teacher remaining nearby. The November gameplay build uses a clearly identified local demo profile until the later enrollment service exists. It must not present an unimplemented voucher screen as a working security boundary.

1.2 Narrative brief

Island services have stopped working. The workshop needs light and supplies; the beacon’s local record display has been corrupted; the assistance center gives unreliable advice. Residents ask the Explorer for practical help.

Sean introduces how to use the island’s digital economy. Matthew helps reconstruct verified records at the beacon. Lee helps train and evaluate an advisory bot. Each restoration visibly changes the world: a workshop lights up, a beacon shines, then a dispatch center resumes supervised service.

This is a helpful adventure, not a combat game. The motivation is restoration and responsibility, not trading profit, punishment for mistakes or competition over wealth.

1.3 Learning objectives

World	Learner should be able to do	Evidence in play
Web3	Distinguish access, ownership and payment; explain a fee; use a conditional program	Review a purchase, authorize it, inspect its record and use owned equipment
Blockchain	Distinguish linked records, validation and agreement; identify stale or altered data	Repair a local sequence from a trusted reference and explain a mismatch
AI	Explain examples, predictions and testing; recognize limits; verify advice	Change training examples, test fresh requests and require evidence before action

Completion is not automatically mastery. A correct answer after a hint is useful practice, but different evidence from an unassisted first response or a fresh transfer question.

1.4 Design pillars and exclusions

Every required concept should support a meaningful player choice, visible system response and understandable explanation. A term may be a tool, station, action, cost or record; not everything should become collectible loot.

Keep journeys short and landmarks readable. Reuse one interaction grammar across worlds: inspect, decide, confirm, see the result and save. Let mistakes invite explanation and retry without permanently losing essential resources.

Excluded from this version: multiplayer, real payments, speculation, live exchanges, public-chain deployment, real seed phrases, live generative chat, procedural open worlds, combat, skill trees, complex NPC schedules and mandatory online student accounts.

2 Gameplay

2.1 Overview

Full-product flow: first-time Teacher or Student selection > authorized setup > Play > world selection > exploration > guide or resident > quest action > feedback > saved checkpoint.

Gameplay-milestone flow: local demo profile > Play > world selection > the same offline quest loop. The temporary profile route is explicitly a demonstration boundary, not a replacement for the full school enrollment requirement.

Web3 opens first. Completing its required chapter unlocks Blockchain; completing Blockchain unlocks AI. Optional encyclopedia discoveries and supporting mini-games do not require 100 percent collection before progression. Completed worlds remain revisitable.

For the milestone, a versioned **demo quest list** defines its shorter chapter. Its completion must be labelled “intro chapter complete,” not completion of every full-world quest. Future content can add new quests without erasing previously earned records or claiming that an unplayed full chapter was mastered.

Core interaction loop

1. See the current objective and location hint.
2. Explore a short route and inspect a relevant object or guide.
3. Receive enough explanation to make a decision.
4. Perform a useful action: pay, verify, repair or test.
5. See what changed and why it succeeded or failed.
6. Commit the checkpoint, then enable the next objective.

2.2 Controls and layout

Landscape orientation is the proposed implementation target. Keep the approved fixed orthographic camera at roughly 35-45 degrees; avoid camera-rotation controls for the milestone.

Input	Action	Required behavior
Left thumb joystick	Move	Stop during dialogue, modal UI, loading or saving transitions
Right contextual button	Interact	Show the nearby target and action; do not activate a distant object
Quest and encyclopedia buttons	Open overlay	Preserve the world and current objective on closing

Input	Action	Required behavior
Tap card, then destination	Place or compare record	Alternative to dragging; selected state uses more than color
Back or pause	Pause or close current layer	Never silently discard a confirmed purchase or completed quest

No jumping, combat, free-camera control or platforming precision is required. Test legibility, thumb reach and safe margins on a named small-screen phone. A proposed 48 dp minimum touch target is a design starting point, not a claim that the current UI meets it.

2.3 Failure and return paths

Insufficient funds show the missing amount and a route to earn the required coins. A wrong answer reveals reasoning and allows retry. An early item remains collected. A cancelled purchase has no effect. A submitted transaction cannot be submitted twice by repeated taps. A save failure keeps the last good checkpoint and offers retry.

After closing and reopening the game, the active objective, completed actions, balances, owned items and restored scene must agree. The success animation alone is not proof that a save succeeded.

3 Game world

3.1 World structure

The full experience has three ordered worlds. For early production they may be three compact scenes assembled from the same island kit, not three large bespoke open worlds. Each scene needs a safe arrival point, a guide visible from the arrival route, a few readable stations and a visible restoration target.

Web3 harbor > Blockchain beacon > AI assistance center. The first world teaches use, the second teaches record integrity, and the third teaches prediction and verification. The same lantern transaction provides continuity across all three.

Location	Main landmarks	Restoration outcome
Web3 harbor	Sean’s arrival plaza, resident, merchant, workshop, exchange and repair terminal	Workshop and harbor services reopen
Blockchain beacon	Matthew’s checkpoint console, damaged local display, verified reference station, stale-node station	Local record repairs and beacon returns
AI center	Lee’s example desk, practice test desk, evidence console, dispatch dock	A supervised helper can assist residents

Place optional discovery objects beside required paths. Use signs, silhouettes and quest markers together; do not make a new learner hunt across the island for the tutorial. Exits explain the specific unmet main objective.

3.2 Web3 harbor



Concept target: a short route from Sean and the resident to the merchant and dark workshop. Scene dressing is illustrative; use the smallest production layout that supports the quest.

The learner should care about the lantern because it enables restoration, not because an NPC demands that the word “NFT” be memorized. Sean links the tool, the payment and the ownership record after the player encounters them.

Web3 quest sequence

Quest	Action and feedback	Scope
W01 Arrive and meet Sean	Walk to the visible guide; open quest log and encyclopedia; inspect the wallet and a shareable receiving address	CORE
W02 Help the resident	Collect one required repair component; return it; receive 12 fictional Chain Coins once	CORE
W03 Buy the lantern	Inspect Lantern #014; review price 8 plus fee 1; sign; see pending then confirmed; balance becomes 3	CORE
W04 Light the workshop	Verify ownership; equip the lantern for free; activate the mechanism; see the workshop restored and save	CORE
W05 Supply the builders	Exchange coins for issued Builder Tokens at a displayed tutorial quote; preserve native fee funds	FULL
W06 Claim the repair agreement	Deliver the required supplies; submit the terminal's completion evidence; call a conditional reward once	FULL
W07 Recover access	Restore a fictional wallet device using its previously secured in-game backup; inspect unchanged ownership	FULL
W08 Reopen the harbor	Combine a new purchase, ownership check and repair evidence in a transfer-of-learning challenge	FULL

For W01-W04 the basic purchase itself illustrates a simple programmed condition: sufficient funds and an available item are required before the atomic exchange. The separate resident agreement in W06 adds external completion evidence and should not be confused with merely choosing "yes" in dialogue.

The resident's 12-coin reward is a one-time quest allocation from the simulation, not a lesson claiming that all cryptocurrency is created by completing errands. There are no optional purchases before this required transaction in the core slice. Later shopping needs a tested resource-recovery route.

Example narrative and teaching beat

Sean: "The workshop needs a lantern. Help our neighbor first, then check what the merchant is asking."

Merchant: "Lantern #014 costs 8 Chain Coins. This transaction also needs a network fee of 1."

Review prompt: "You have 12. How many remain after this transaction?" A wrong answer highlights the price and fee separately, then permits another try without charging the player.

After confirmation, Sean asks the learner to open the ownership record and find the lantern. At the workshop, the learner equips it without a new payment. A final new example asks whether reading the encyclopedia also costs gas. The explanation identifies which actions change the simulated network state.

A correct mathematical answer alone is insufficient: the learner must also see the item and payment change together, and distinguish the ledger record from the wallet interface.

3.3 Blockchain beacon



Concept target: repair one damaged local station using a trusted reference. Other stations represent local simulated copies on the same phone, not connected student devices.

The beacon's local copy is damaged. Matthew can access a trusted checkpoint and verified reference history. The learner repairs the local display and record; they do not rewrite accepted global history.

Blockchain quest sequence

Quest	Action and feedback	Scope
B01 Read the familiar record	Inspect the confirmed Web3 purchase, including 8-coin price, 1-coin fee and Lantern #014 transfer	CORE
B02 Restore the sequence	Place three subsequent display blocks after a trusted checkpoint by their previous-block references	CORE
B03 Reject the altered copy	Compare a changed 6-coin local payment with the verified 8-coin original; identify the broken reference	CORE
B04 Synchronize a stale node	Load verified missing blocks into a behind-but-not-tampered copy	FULL
B05 Help validate a batch	Check authorization, funds, ownership and repeated transactions before a simulated network decision	FULL
B06 Compare mining and validators	Operate a bounded nonce-search demonstration; contrast it with the main validator workflow	FULL
B07 Restore the beacon	Recheck the local checkpoint, restore the light and explain the repair	CORE for demo; FULL synthesis later

B02 and B03 use one bounded scenario, not a general block editor. The verified reference remains read-only. Blocks may contain multiple records. Timestamps and colors may aid reading but cannot be the reason a sequence is accepted.

An out-of-date copy is not necessarily dishonest. A changed copy that breaks a trusted reference is a different fault. If no trustworthy reference exists, the correct explanation is that the original is not established, not “the player chooses which history is true.”

3.4 AI assistance center



Concept target: a teaching desk, separate testing area and supervised dispatch station. The learning model and later assistant are distinct systems.

Lee’s helper examines fictional transaction requests. Blockchain checks whether a transaction meets network rules; the helper offers fallible advice about whether a person should review the request. A valid signature does not prove that a request is wise or honest.

AI quest sequence

Quest	Action and feedback	Scope
A01 Teach with examples	Read contextual evidence; label a small example set Routine or Review; fit an explainable tiny classifier	STRETCH
A02 Test unfamiliar requests	Predict what the bot will say; run new cases; inspect mistakes; correct an example and practice again	STRETCH
A03 Make the final check	Use a reserved fresh set; explain one limitation; restore the teaching desk	STRETCH
A04 Ask and verify	Build a request with goal, context and constraints; check an authored assistant response against the ledger	FULL
A05 Supervise dispatch	Inspect a proposed recipient, item and cost; approve or reject; recheck changed conditions	FULL

The recommended first implementation is a small nearest-neighbor classifier over explicitly defined features, not a neural network, cloud chatbot or financial fraud engine. It must actually depend on the learner's labelled examples. If outputs are authored branches, label the activity an AI demonstration instead of a trained model.

Use synthetic requests only. Include a legitimate large community purchase and a suspicious small request so "large amount means bad" is visibly inadequate. An unknown case can require human review. The bot never spends without approval.

A01-A03 can form a short standalone intro chapter after the core passes. A04-A05 add modern assistant and agent concepts later; they are not required to make the first classifier lesson educationally honest.

4 Graphics and art style

4.1 Art direction and production boundaries

Retain the approved fixed isometric, restrained low-poly direction. Use faceted silhouettes, angular ruins, simple material surfaces, sparse painterly detail, readable long shadows and controlled glow. Water and terrain should frame paths rather than create visually busy routes.

The approved Explorer has blue hair and yellow clothing; friendly robot guides use white bodies and dark screen faces with cyan expressions. Apply the same faceted treatment to characters and environments. Do not return to the superseded rounded toy or bright voxel-diorama look.

Tunic is a camera, material-density and composition reference only. Do not copy its maps, characters, props, icons or HUD. The supplied Project Vanishing document controls the GDD's organization, not Chainy's camera, genre or art assets.

Minimum production kit

Asset family	November candidate	Later expansion
World pieces	Shared ground, wall, path, stair, bridge, water and foliage pieces	More regional dressing after performance checks
Interaction stations	Merchant, workshop mechanism, ledger console, beacon	Exchange, mining, recovery and AI dispatch stations
Characters	Existing Explorer plus one reusable robot rig with named variants	Extra resident silhouettes and reactions
Inventory	Repair component, lantern, wallet icon and coin symbol	Builder Token, supplies and contract evidence icons
Feedback	Objective marker, selection outline, success/error symbol, restoration light	Extra cosmetic animation only after core acceptance

Luis provides source files, export previews, scale/origin notes, materials, animation lists and provenance. Miguel implements interactive UI. Christian validates the imported scene on Android. Final polygon counts, texture sizes, memory and package budgets follow a measured representative scene; a 4 GB RAM phone does not give the game all 4 GB.

4.2 Interface design

The exploration HUD shows one short objective, quest/encyclopedia access, movement and interaction. Keep additional panels contextual. The wallet has Balances, Items and Activity views. Display native Chain Coin separately from Builder Token and unique equipment.

Dialogue remains over the live isometric scene: wide bottom panel, Explorer portrait on the left, speaker portrait on the right and centered choices above. Disable movement and world interaction while the overlay is active. Neutral choices must not reveal the correct answer through color before selection.

All essential feedback works with sound off and includes text or an icon as well as color. Let learners control dialogue pacing, reopen explanations and use tap-to-place instead of dragging. Keep decorative hex strings secondary to understandable sender, recipient, amount and status fields.

4.3 Storyboard and GUI development



Three separate screen studies, not three panels shown simultaneously on a phone. The quest log separates a current goal, its next action and completed history; encyclopedia stages record learning activities rather than a mastery percentage.

Panel	What the player sees	What the system must do
1 Play and worlds	Web3 available; Blockchain and AI have specific unlock labels	Resume the active demo checkpoint or begin the tutorial
2 Arrival	Sean visible near a short path; one objective	Teach movement and interaction without a long search
3 Explain	Live scene behind the dialogue overlay	Capture only dialogue input; add the basic encyclopedia entry
4 Review purchase	Item, price, fee, total and remaining balance	Permit cancel; reject invalid input before submission
5 Confirmed result	Lantern ownership and activity record; new balance	Commit related state once before showing saved success
6 Restoration	Workshop illuminates and route updates	Award completion once; unlock the demo's next chapter
7 Record repair	Trusted reference beside selectable block cards	Explain the specific mismatch; preserve valid progress on retry
8 AI introduction	Training cards and a separate fresh test	Show an actual prediction, its limitations and a human decision

These panels define states, not eight separate scenes. Reuse overlays and the same world wherever possible. A visually finished screen without its cancel, retry, error and resume paths is not a finished interaction.

Quest log and encyclopedia

The log presents completed, current and locked main objectives chronologically. The active entry includes its resident, place, next action and progress. Optional discoveries are visually separate from required progression.

The encyclopedia uses four records: Discovered, Explained, Practiced and Demonstrated in a new situation. None is named “mastered.” Discovery adds a basic entry immediately. Identification adds meaning and a concise contextual prompt. A wrong answer must not hide the reference needed to learn.

No quiz appears after each coin or duplicate item. Group related discoveries and let the player consult entries in the field. Each entry includes what the concept does, an island example, one common misconception and the relevant activity.

4.4 Sound design

Use quiet island ambience, light mechanical cues and clear confirmation sounds. Give purchase review, confirmation, invalid action and restoration distinguishable sounds without alarm-heavy punishment. Music should not compete with reading.

The milestone needs a small licensed or original sound set, volume controls and a silent-play path. Voice acting, adaptive scores and unique tracks for every station are not required. Document the license and source of every shipped sound; concept images do not authorize third-party audio reuse.

5 Mechanics

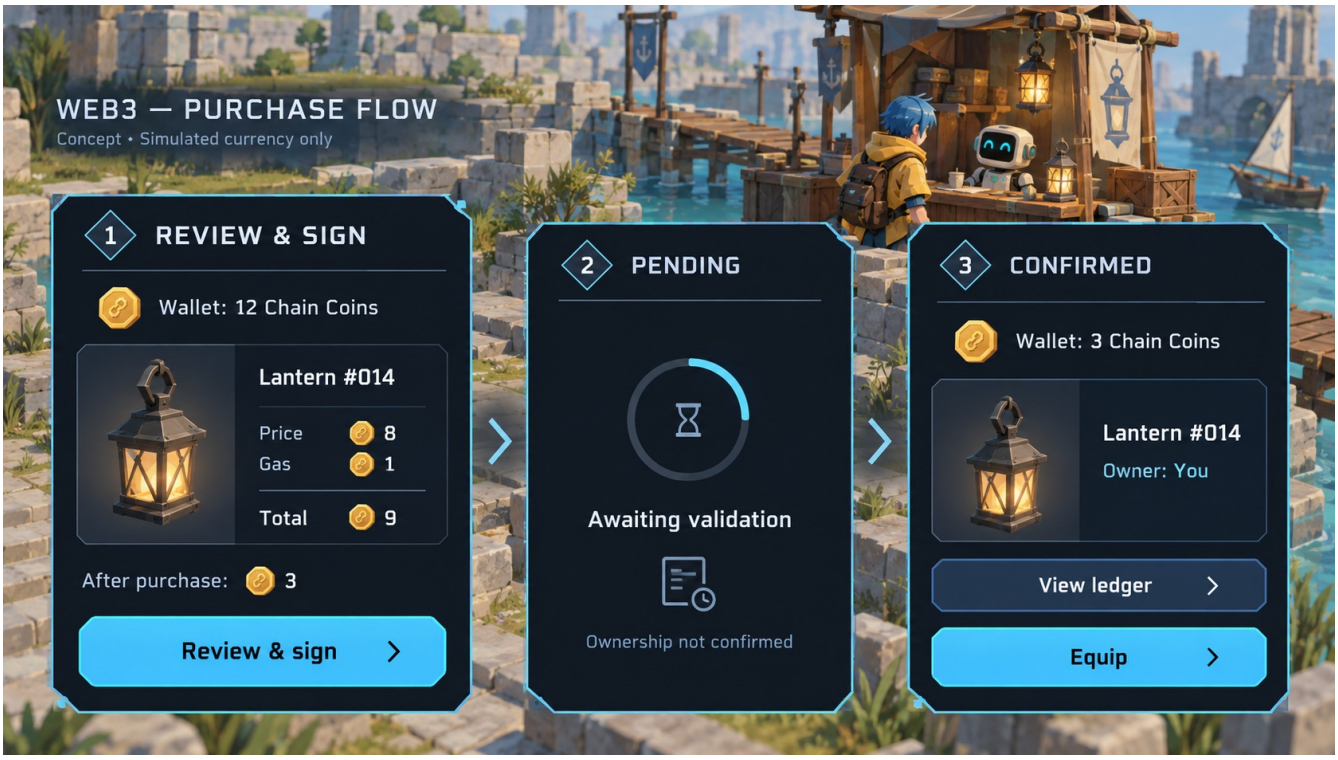
5.1 Quests and dialogue

Quest states are Locked > Available > Active > Ready to report > Completed. A quest can skip Ready to report when its completion occurs at a station. Each transition has a prerequisite, an action, a success response and a save boundary.

Situation	Required rule
Early pickup	Record it once and count it when the relevant quest becomes active
Repeated report	Replay feedback if helpful; never repeat a one-time reward
Wrong answer	Show a hint or counterexample; retain the item and allow retry
Modal open	Suspend world input so one tap cannot both answer and move
Interrupted scene	Reload the last committed quest state and reconstruct world changes
New content	Preserve completed objective/version records; explicitly map the next required quest

Dialogue should explain a decision in short pieces, not deliver an entire chapter before the player can act. Narrative choices may change feedback, but branching relationship systems and punishments for offending NPCs are outside scope.

5.2 Wallet and purchase



Reference sequence: 12 coins, price 8, fee 1, total 9, remaining 3. Generated UI labels illustrate hierarchy; the rules below govern the implementation.

The wallet is an interface to account information and authorization. The ownership record belongs to the simulated ledger, not physically inside the wallet picture. An address is shareable receiving information; secret signing material is not.

Purchase states: Inspect > Review > Authorize > Pending > Confirmed or Failed. Review includes the recipient, item identifier, price, network fee and total. Cancelling before submission changes nothing. Confirmation changes balances, item ownership, activity and quest state atomically and once. Equipping an owned lantern does not submit another payment.

The milestone uses toy authorization checks clearly labelled a simulation. It does not need real seed generation, cryptographic signing, a public node or a blockchain SDK. If production later implements real hashes, use full hashes internally while shortening only their display. [S3-S5]

5.3 Economy and ownership rules

Chain Coin is the fictional native currency used for the illustrated network fee. Builder Token is a fungible asset issued within that simulated network. Lantern #014 is a unique item whose token ownership is recorded; a separate receipt describes a transaction. Not every stackable supply needs an NFT. NFTs do not guarantee investment value or transfer every creative right. [S3, S4, S6]

For the first purchase, the learner starts with 12 Chain Coins and spends exactly 9 including the fee. Failed review checks charge nothing. Later lessons may illustrate an executed failed transaction consuming a fee, but must distinguish that from a pre-submission rejection. Movement, reading, viewing balances and ordinary equipment use are free. [S4]

The full exchange uses a displayed fixed teaching quote, with input, output, fee and post-trade balances. Show that swapping assets differs from buying a tool. Prevent a required quest from becoming unwinnable because all fee currency was exchanged. No speculative prices, scarcity manipulation or real-money purchase is needed.

Smart contract interaction

A full repair agreement exposes the condition, trigger and result: “Required supplies received AND valid workshop completion evidence submitted AND reward not already claimed.” The player invokes the claim. A completion terminal supplies the in-game evidence; the program does not magically observe a real bridge or workshop.

An unmet condition explains what is missing. The condition is rechecked at commit time. A repeated claim returns Already claimed without another reward. A purchase can use a simpler conditional program before this broader quest is built. This is a teaching simulation, not executable Solidity or a deployed contract. [S5]

Seed phrase recovery

In the full version, Sean helps secure a game-only backup before a later recovery exercise. Never scatter secret words as public treasure or ask the learner to enter a real wallet secret. Use an unmistakable “Practice backup - not a real wallet” screen and bounded fictional symbols or words.

Recovery restores access to the same simulated account and its existing ownership records; it does not mint a second lantern or reset balances. The fictional backup is unrelated to the student voucher, learner identity, teacher PIN or actual device save. A lost phone’s unsynced progress cannot be recovered by this lesson mechanic.

The first milestone can explain backup safety through dialogue, but that does not count as an implemented hands-on seed-recovery mechanic.

5.4 Blockchain inspection and repair



Reference interaction: inspect a trusted checkpoint, link the display blocks and reject changed data. The plate shows two placement slots for readability; the proposed core scenario uses three subsequent blocks. A/B/C are symbolic reference labels, not cryptographic hashes.

The core repair uses a known starting checkpoint and three subsequent display blocks. Each block exposes its identifier, previous reference and a concise record summary. One record refers to the actual lantern

purchase. One block may contain several transactions, so the interface must not teach a universal one-transaction-per-block rule.

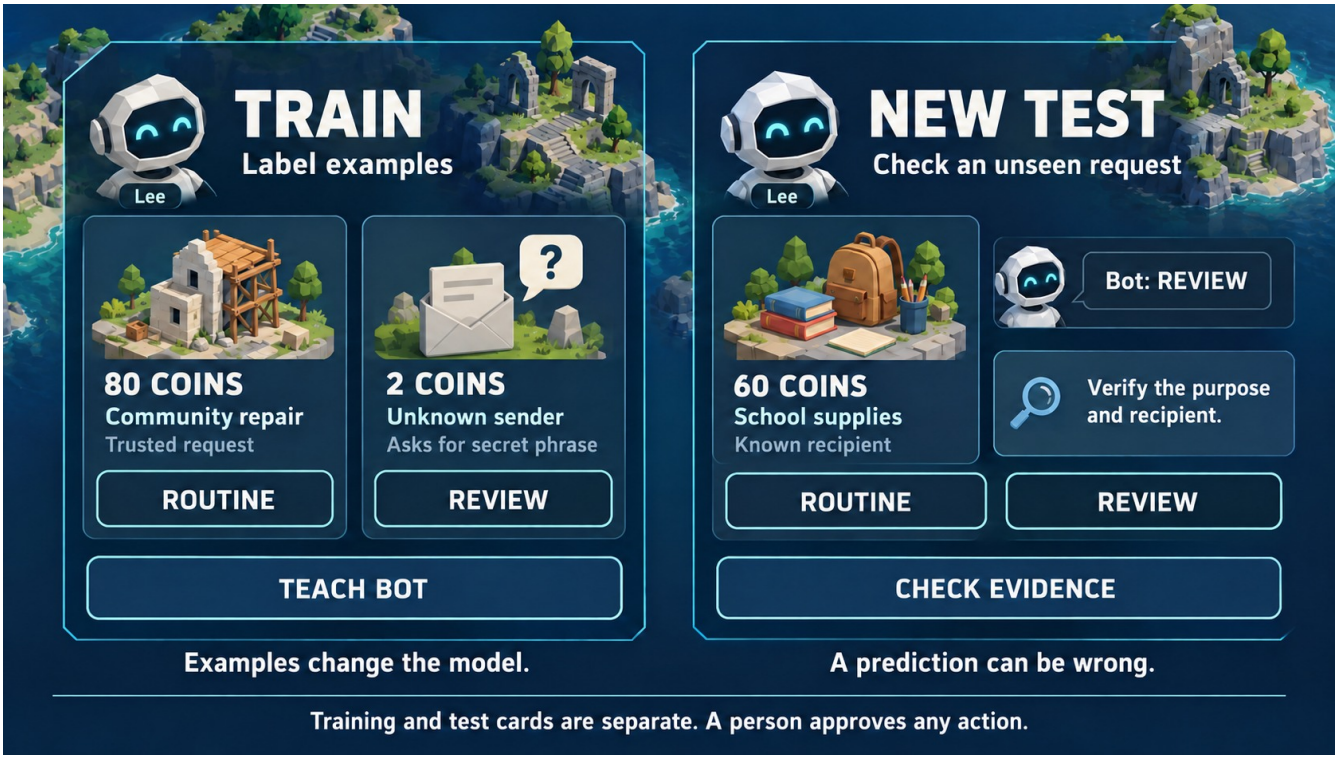
The learner selects a block, inspects its evidence and places it. A mismatch explains which reference failed. In the tamper case, the damaged display changes the price from 8 to 6; the trusted record still says 8. Restore from the verified source rather than asking the learner to arbitrarily edit the accepted payment.

The full version adds stale-node synchronization and candidate-transaction checks for authorization, balance, ownership and replay. A network decision is simulated separately; one player clicking Approve is not real consensus. Nodes and validators are related roles, not synonyms. [S7, S8]

The mining station is a separate proof-of-work comparison: vary a nonce, search for a hash meeting a toy target and show that verification is quicker than the search. Bound the demonstration and CPU work. Do not turn mining into hitting a rock or imply that the main proof-of-stake network also mines its blocks. [S8, S9]

The beacon's restoration condition is a valid local reconstruction against the scenario's trusted reference. It is not a claim that every real node must always be online and identical at the same instant.

5.5 AI teaching and testing



Reference interaction: separate labelled examples from new test cases, show advisory predictions and preserve human review. The left panel shows labels already selected; the playable version must offer both choices first. The panels are separate screens, not simultaneous phone UI. Artwork does not establish model accuracy.

Use a small, curated synthetic dataset. Proposed features are known recipient, amount relative to usual activity and permission type. Each card also has story evidence so the learner is not labelling blindly. Labels mean Routine and Needs review, not guaranteed safe and criminal.

A candidate three-nearest-neighbor model compares normalized features and takes a label vote. Define ties, missing features and very unfamiliar cases as Review. The exact feature encoding and distance rule need a small prototype; the acceptance requirement is that changing relevant training labels or examples can change predictions on held-out requests. Do not display a vote fraction as a calibrated confidence percentage.

A proposed teaching set contains six examples, a separate four-case practice set and a reserved four-case final set. These counts are content-production starting points, not statistical validation. Practice feedback can guide corrections; final-set labels must not influence training before the recorded first final attempt. Replays become practice and must not be described as fresh independent tests. [S10]

Include counterexamples with sufficient context: a large approved workshop purchase, and a small request for inappropriate permission. A fresh final scenario should test the learner's reasoning, not whether the bot can be forced to score perfectly.

For the later assistant activity, bundle authored responses and clearly label them simulated. A learner supplies goal, context and constraints, then opens records to verify a plausible answer. Prompt context is not retraining, and an AI text token is not a blockchain asset token. Later dispatch is a bounded scripted agent workflow: observe, propose, ask permission, act and inspect. It is not evidence that ordinary pathfinding has learned a model. [S11]

5.6 Coverage of all twelve requested concepts

Concept	Full hands-on mechanic	November boundary
Cryptocurrency	Earn and budget fictional Chain Coin	CORE reward and purchase
Blockchain	Inspect own record and reconstruct local sequence	CORE bounded repair
Exchange	Preview and swap coin for Builder Tokens	FULL; explanation is not hands-on completion
Gas fee	Review fee separately from price	CORE purchase; advanced failure-fee lesson later
Keys	Share address; authorize with protected signing control	CORE toy signing interaction; not real key management
Mining and validators	Compare nonce search with distinct candidate-validation workflow	FULL; only basic verification in core
NFT	Obtain numbered equipment and inspect ownership	CORE Lantern #014
Smart contract	Inspect a purchase condition; later invoke a repair claim	CORE simple purchase condition; FULL external-evidence claim
Token	Spend issued Builder Tokens distinct from native currency	FULL
Wallet	Check balance, items, activity and authorization	CORE compact wallet
Seed phrase	Recover access with a previously protected fictional backup	FULL
Node	Compare and synchronize local simulated copies	CORE reference copy; FULL stale-node repair

This table is a scope map, not a "concepts completed" percentage. Some concepts have an introductory action in the core and a substantial later exercise. Counting all twelve as complete because their names appear in a menu would misrepresent the result.

5.7 Supporting quizzes and mini-games

Quiz prompts follow a relevant explanation or action. Use one concise identification question, a decision during use, and a new example at a milestone. Incorrect answers give explanations without taking essential coins. Record first response, hint use and eventual success separately when that evidence is needed.

Wordle supports vocabulary and definitions; it is not a substitute for using a concept. Its dictionary, word length, repeated-letter feedback and attempts require a rules sheet. BlockMatch remains the established matching/cascade activity until its own objective and educational relation are approved. Do not silently replace it with the new ledger-ordering exercise.

Quiz can reuse the dialogue/choice interface in the core. Standalone Wordle and BlockMatch remain in the full design but are deferred from November 10 by the owner's October 23 decision. This project-scope decision is not evidence that the school has waived any requirement; Sean must record the school's actual expectations. Completion of a spelling or matching puzzle must not be reported as proof of understanding blockchain or AI.

5.8 Local state and implementation boundaries

Only one activity owns gameplay input at a time: exploration, dialogue, purchase, record repair or AI exercise. Opening a modal releases movement. A pending commit blocks repeated confirmation; closing a panel does not cancel a transaction already committed.

Reuse the existing controller and dialogue. Reconcile new quest, simulation and save responsibilities with Christian’s actual Game Manager, Progress Manager, Database Manager, event and scene interfaces. Do not begin with a replacement framework or new service platform.

SQLite remains the local store. Minimum persistent gameplay state includes profile ID, build/content/schema versions, active/completed quests, discovered entries, inventory and balances, ledger state, world restoration flags and relevant activity checkpoints. These are proposed record needs, not verified table definitions.

For a purchase or reward, balance, ownership, quest and ledger changes must succeed together. Use a stable action identifier to make duplicate taps and repeated resume handling harmless. Confirm “Saved” only after the database confirms success. Test interrupted commits, no-space failures, force-close and relaunch with synthetic profiles.

Data-heavy systems are deferred, but postponing save design until after all quests would be a delivery risk. The smallest reliable save is part of the first complete loop.

5.9 Later teacher and distribution service

The full product retains Teacher/Student entry, developer-issued teacher vouchers, one online teacher activation, offline student pairing, a protected dashboard and local collection for approximately 20-40 personal student phones. Teachers later upload individual learner-coded results; names and the identity mapping remain with the teacher.

Separate three payloads: logical learning records, compatible content packages and signed Android app updates. They require different validation, retry and recovery rules. A teacher dashboard shows last received data, not live progress while phones are disconnected. Student devices are not nodes in the fictional blockchain lesson.

Nearby Connections or a local Wi-Fi session are candidates requiring device prototypes. No promise is made that 40 phones connect simultaneously, that updates are silent, or that SQLite itself provides synchronization. These capabilities are outside the November gameplay commitment. [S12, S13]

Do not collect real learner records in an improvised developer service. School authorization, receiving account, purpose, retention and deletion rules remain prerequisites. Demo tests use synthetic codes. A data-collection or school-evaluation obligation must be resolved with the adviser, not silently removed because services are deferred.

6 Characters



Proposed character family: the Explorer and three variants of a reusable robot body. Names and Explorer identity are established; guide badges and accent colors remain design options. Match production proportions consistently across all scenes.

6.1 Explorer

The Explorer is the player-controlled blue-haired character in yellow clothing. Use the existing movement and dialogue assets where suitable. The character helps residents and gains access to tools and restored locations, not combat power or speculative wealth.

Minimum animation states are idle, walk, interact and a short completion reaction. Equipment such as the lantern should attach consistently to the existing rig. Movement, interface focus and equipment state must agree when a dialogue closes or a scene resumes.

6.2 Guides and residents

Character	Function	Distinguishing visual cue	Teaching behavior
Sean	Web3 arrival and economy guide	Wallet/book badge, warm station accents	Explains a practical need before introducing terminology
Matthew	Blockchain beacon guide, formerly Blocky	Linked-block badge, cyan record station	Asks for the reference or evidence behind a repair
Lee	AI assistance guide, formerly Aira	Example-card badge; a distinct restrained accent	Invites predictions, tests and admission of uncertainty
Residents	Give restoration requests and consequences	Reusable silhouettes and context props	Explain why the requested action matters

Guide appearance variants are proposals for owner review. The guide names are confirmed. Reusing the existing friendly screen-faced robot base reduces model, rig and animation work; a new human character for every name is not assumed.

The NPC Sean and the team member Sean are different roles. Documentation ownership does not imply a character portrayal.

6.3 Character scope

No combat animation, hostile reputation, daily routine simulation, creature ecology or elaborate pathfinding is required. Residents can remain at readable interaction points. A dispatch bot can use a short scripted path later without being presented as the learning model itself.

Dialogues should be encouraging but specific: identify the mistaken assumption and explain the correction. Avoid calling a learner unintelligent, punishing an incorrect answer with lost progress, or treating a prediction as certain because a friendly robot says it.

7 Project plan and November 10 feasibility

7.1 Review of the previous output

The earlier proposal is coherent as a full-product direction: the Web3 action creates a meaningful record, Blockchain makes that record inspectable, and AI teaches judgment over its use. The best parts to preserve are visible restoration, repeated use of the same lantern transaction, evidence-based repair, and the distinction between practice and independent learning evidence.

Its main delivery problem is breadth. It adds a quest runtime, inventory, simulated economy, safe transactions, ledger repair, training/test activity, several new UI families and dependable persistence, while also proposing school authorization, peer transfer, analytics, content distribution and upload. Those are not all supplied by movement, dialogue or SQLite.

The owner's latest evidence is **movement and dialogue only**. Earlier repository statements about other components being in progress are not treated as playable acceptance evidence. The actual Unity version, current revision, Android APK, database integration and representative phones still need verification.

Verdict

A substantial gameplay slice by November 10 is plausible but high risk. “At least 50 percent of the entire earlier proposal” is not a defensible commitment today. Scope must be fixed and tested before a percentage can be claimed.

Adopted core commitment: W01-W04 plus B01-B03 and the demo B07 restoration, with local saves, contextual Quiz prompts and a compact Android UI. Conditional stretch: A01-A03 using the same card/panel components. Retain exchange, Builder Tokens, full contract claims, seed recovery, mining/validator comparison, AI assistant/agent work and standalone Wordle/BlockMatch for later gates.

This is a genuine change from “all twelve concepts deeply playable now.” It preserves them in the full design without claiming the smaller milestone implements them all. If an adviser requires three playable subject modules, move the small AI chapter into required scope and reduce Web3/Blockchain content or renegotiate the target; do not quietly show concept art as a functioning AI module.

7.2 What 50 percent would require

For an internal gameplay percentage, freeze the full deliverable baseline and agree weights for functional systems, content depth, usability and verified quality before evaluating the build. Record accepted evidence against that baseline. Do not choose weights afterwards to make the result reach 50.

For the school’s requirement, obtain its actual calculation and named criteria. The documented 60 percent pre-final requirement and eight-criterion quality rating cannot be converted into a completion percentage by counting chapters or tasks. November 10’s requested 50 percent is therefore an aspiration, not a demonstrated school result.

The concrete demo below remains useful even if no percentage is approved: earn > buy > confirm ownership > restore > repair a local record > save > relaunch. It lets the team demonstrate working behavior instead of an unverifiable number.

7.3 Capacity and effort estimate

There are 19 calendar dates from October 23 through November 10 inclusive, or 13 weekdays. The developer’s stated maximum of 16-18 hours per day gives 304-342 nominal available hours across all 19 dates. This is arithmetic, not a forecast of productive output; it excludes interruptions, school/travel commitments, integration, rework and recovery.

Plan the first pass around 6-8 focused technical hours per available day and protect testing time. On all 19 dates that is 114-152 hours; on weekdays alone it is 78-104 hours. The November 10 community beta and its preparation consume availability and must be subtracted from these figures. UI/art availability has not been provided.

The following are order-of-magnitude estimates, not measured productivity or commitments. They assume the existing movement/dialogue work on Android, access to the actual source, reusable UI, simple placeholder art, one local profile, a small deterministic simulation and no networking. They include developer integration of separately supplied UI/art, but not creation of all production assets.

Work package	Focused lead-dev hours	Dependencies and completion evidence
Android baseline and state/save scaffold	10-16	Named APK; synthetic profile survives exit and relaunch

Work package	Focused lead-dev hours	Dependencies and completion evidence
Quest, interaction and inventory loop	12-20	Pickup, report, one-time reward and scene state reload correctly
Wallet, purchase and workshop restoration	18-28	Exact fee arithmetic; atomic ownership/payment; no duplicate reward
Bounded block ordering and altered-record case	12-20	Trusted-reference explanation and recoverable wrong placement
Shared UI/content integration and unlocks	8-14	Usable panels; demo quest list; no circular gate
Android regression and recovery testing	16-24	Save/resume, interruption, repeat-tap and low-device checks
Core subtotal	76-122	W01-W04 and bounded Blockchain demo
Contingency at approximately 25 percent	19-31	Unknown code condition and rework, not extra features
Core planning envelope	95-153	Conditional target; re-estimate after the first accepted loop
AI training/test stretch including contingency	20-35 additional	Reused card UI, data-dependent model, separate fresh final cases

Indicative parallel supply needs: 12-24 focused hours for UI layouts/assets, 12-24 for a reusable art kit and scene dressing, and 8-16 for reviewed instructional copy and QA cases. These are separate lane estimates, not confirmed team availability, and cannot be added to Christian's implementation capacity. If those handoffs are unavailable, the technical path expands or visuals remain provisional.

The 95-153-hour core overlaps the 114-152-hour all-days planning envelope but exceeds much of the weekday-only envelope. Thus the core is conditional, and the AI addition is clearly high risk. This comparison is more useful than assuming 16-18 hours every day guarantees success.

7.4 Milestones and dependency gates

The owner confirmed on October 23 that beta moves to November 10 as Cryptita Plays community beta testing; pre-final remains November 21. Internal dates below are planning targets, not measured capacity or evidence of school approval.

Window	Priority and gate	Failure response
Oct 23-24	Verify source/Android build; one pickup, reward, save and reload	Stop expansion; repair foundation before more screens
Oct 25-28	Complete W01-W04 with temporary art and exact transaction rules	Remove optional content; keep the complete restoration loop
Oct 29-30	Integrate bounded Blockchain repair only if Web3 is stable; freeze beta candidate	Ship a clearly limited Web3 demo if repair is not ready

Window	Priority and gate	Failure response
Oct 31-Nov 2	Device tests, instructions, known issues and synthetic rehearsal	Fix blockers; no new gameplay subsystem
Nov 3-6	Integration, target-device checks and recovery testing	No automatic scope expansion from the moved beta
Nov 7-9	Regression, beta rehearsal and candidate freeze	Fix blockers; no new gameplay subsystem
Nov 10	Cryptita Plays community beta testing	Use the tested build; collect only approved evidence and disclose missing scope
Nov 11-13	Triage findings, retest fixes and analyze actual evidence	Do not invent research results or add broad features
Nov 14-16	Complete manuscript and verified evidence package	Separate community feedback from school-required research evidence
Nov 17-20	Rehearsal, recovery drill and release freeze	Fix only release-blocking defects
Nov 21	Pre-final demonstration	Use the tested build and disclosed limitations

This schedule replaces missed internal targets; it does not claim earlier October milestones were achieved. Any failure of the core gate requires an explicit scope/readiness decision, not silent removal of tests or rehearsal.

Critical dependency chain

Verified Android build > reliable quest save > complete Web3 purchase/restoration > shared ledger data > bounded Blockchain repair > optional AI lesson > integrated regression > demonstration.

Art, copy and UI may proceed in parallel after the interaction fields and states are frozen. Multiple contributors do not remove the single-developer integration bottleneck. AI-generated designs reduce illustration time; they do not implement Unity interactions or validate learning outcomes.

Stop rules

If no saved end-to-end quest works by October 28, reduce to one polished restoration quest and replan. If transaction duplication, save loss or progression dead ends remain, do not begin the AI stretch. Do not spend the final test window adding exchange, teacher networking or a new UI framework.

7.5 Ownership and handoffs

Lane	Owner	Required output to the next lane
Scope and learning-game decisions	Sherwin	Approve core boundary, mini-game deferral, date changes and milestone interpretation
Game code, persistence, builds and integration	Christian	Named revision/APK, state contract, reproducible technical tests
Art assets	Luis	Reusable imported kit, source/provenance and scene previews
Authored project documentation and learning copy	Sean	Approved quest text, concepts, questions, acceptance cases and manuscript links
Game UI implementation	Miguel	Panels and input/error states conforming to the technical interface

Each work card has one accountable owner. Separate the UI handoff from code integration and the design decision from document authorship. This edition proposes work packets; it creates no issue, changes no task status and publishes no board changes.

Review findings from the project skills

Review lens	Result applied in this edition
Scope and dependency planning	Core, stretch and later-service boundary; capacity ranges and explicit dependency gates
GDD requirements and learning design	Observable actions, responses, retries and learning checks rather than definition-only fetch quests
Simulation accuracy	Local ledger repair, distinct mining/validation, advisory AI, fictional secrets and no real funds
Architecture and Unity delivery	Reuse existing movement/dialogue; verify actual Android build; define state/save ownership first
Gameplay implementation	Deterministic bounded scenarios and duplicate-safe actions; no code changes implied by this design
Asset and UI pipeline	Reusable kit, fixed isometric presentation, mobile overlays and concept-versus-production distinction
Android performance	Named-device profiling before support claims; preserve QA time
Offline evidence	Keep local saves now, defer transfer/upload; learner records separate from fictional transactions
QA, defense and release	Reproducible checks, truthful scope reporting, preserved pre-final and handover requirements

The fourteen named project skills informed these decisions; they are review lenses, not independent human approvals. The visual swarm supplies concept assets only. Sherwin still approves scope, Christian validates implementation facts, and Sean reconciles the adopted edition into project documentation.

7.6 Acceptance and device checks

ID	Test	Passing result
GP01	Start a new local demo profile offline	Web3 opens; other demo chapters have clear gates
GP02	Open dialogue while moving	Movement stops; closing returns control without an unintended action
GP03	Collect an item early, answer incorrectly and return	Item persists; explanation remains accessible; no progression dead end
GP04	Earn the resident reward twice	Exactly 12 coins are granted once
GP05	Buy Lantern #014 from 12 coins	Price 8 plus fee 1 leaves 3; ownership and record agree
GP06	Cancel review or double-tap confirmation	Cancel changes nothing; repeated confirmation commits at most once
GP07	Equip the lantern and restore the workshop	No extra gas; restoration and quest completion save together
GP08	Force-close and relaunch after checkpoints	Balance, inventory, records, quests and restored scene agree
GP09	Simulate a save failure or insufficient storage	No false Saved message; last good state retained; recovery offered
GP10	Place blocks by color/time rather than valid references	Invalid sequence rejected with a precise explanation
GP11	Compare altered 6-coin payment with verified 8-coin source	Learner repairs local copy; cannot overwrite trusted source
GP12	Finish demo Web3 and Blockchain chapters	Correct next demo chapter unlocks; completed worlds remain accessible
GP13	Train, change examples and test new AI cases	Outputs depend on examples; practice and final records remain distinct
GP14	Reject unsupported AI advice	No automatic spend; evidence and human review remain available

GP13 applies only if the AI stretch is delivered; GP14 belongs to the later assistant/dispatch release. “Not implemented” is a valid status and must not be replaced with a fabricated pass. For executed checks, record the build, content version, device, steps, expected/actual result and evidence.

The device specification remains a target: 4 GB physical RAM, 64 GB total storage, ARM64, OpenGL ES 3.2 or Vulkan 1.1 and an entry-to-midrange GPU. Android 11+ is the provisional test interpretation; Android 10 compatibility remains unverified. Total storage is not the app’s allowed footprint.

Proposed performance target: a readable, responsive scene around 30 fps on the selected lower-end test phone. Measure loading, sustained scene performance, memory, heat, suspend/resume and storage use

before publishing support claims. Verify permissions, ARM64 packaging, signing and save-preserving updates on real hardware.

Demonstration and release evidence

Prepare one named APK, a short installation guide, synthetic demo profile, known-issues list and reproducible restoration demonstration. Show offline play and a relaunch that preserves the result. Keep signing credentials outside repositories. Do not present the teacher dashboard, generated art or future service diagrams as a live feature.

8 References and template adaptation

8.1 Main structural reference

The supplied 27-page *Games Design Document - Project Vanishing* is the main organizational reference. Its structure is applied as follows, with PDF page numbers counted from the first page:

Reference pages	Reference element	Chainy adaptation
4	Game, narrative and objective briefs	Section 1 defines the educational restoration adventure
5	Gameplay and control layout	Section 2 uses mobile touch and fixed isometric controls
6-7	World map and location descriptions	Section 3 defines three compact, ordered worlds
7-13	Art, GUI, storyboard, sample narrative and sound	Section 4 plus W03 example dialogue; concept sheets illustrate interfaces
13-18	Quest/dialogue, progression, AI/pathing and economy	Section 5 substitutes educational systems and explicitly excludes combat/reputation
19-21	Character mechanics, development and animation	Section 6 keeps Explorer/guides and minimal reusable animation
22-26	Project plan, Gantt and sprint reports	Section 7 gives dated gates, dependencies, estimates and acceptance evidence
27	References and bibliography	Sections 8-9 identify the design and technical sources

The reference's table of contents and body numbering diverge around Mechanics: Weapons appears as 5.2 in the body, while later systems shift. This edition uses its own consistent numbering. It does not reproduce the sample's "BREIF" typo, control-direction inconsistency or example student progress reports as Chainy facts.

The source's combat, hostility, class development, first-person perspective and medieval story are examples, not instructions to add them to this educational mobile game. No source artwork is copied into the new visual pack.

8.2 Project evidence and limitations

Direct owner decisions govern this edition; Section 1 and the later-service boundary retain them. The latest report of movement and dialogue being playable supersedes older “in progress” descriptions. Project sources are listed under S2. No APK, performance or learner test was performed for this document; estimates are not measured results.

9 Bibliography

Design and project sources

S1. Supplied *Games Design Document - Project Vanishing*, 27-page PDF, inspected October 22, 2026. Main structure and presentation reference; not a Chainy implementation source.

S2. Chainy project documents and owner decisions, inspected October 22, 2026: current baseline, GDD, technical design, requirements, active plan, hardware validation, art/UI brief, offline-data plan, learning and evaluation documents; plus *Chainy Gameplay Validation and Proposed Journey* from this conversation. Version conflicts are resolved in favor of the latest direct owner decision.

Primary technical sources

S3. Ethereum documentation, Accounts. Supports account, address, key and wallet distinctions; Chainy's authorization remains fictional.

S4. Ethereum documentation, Gas and fees. Supports transaction-cost distinctions; Chain Coin and the 1-coin tutorial fee are authored game rules.

S5. Ethereum documentation, Smart contracts. Supports triggered program behavior and external-information limits; no deployed contract is required.

S6. Ethereum documentation, Non-fungible tokens. Supports unique token/ownership distinctions; the lantern is an educational example.

S7. Ethereum documentation, Nodes and clients. Supports node roles; classroom devices are not used as actual blockchain nodes.

S8. Ethereum documentation, Proof of stake. Supports the distinction between validator participation and a single learner's approval.

S9. Ethereum documentation, Proof of work. Supports the separate bounded mining comparison.

S10. Google Machine Learning Crash Course, Training, validation and test sets. Supports separating practice and fresh tests; a tiny teaching dataset does not establish real-world reliability.

S11. Google Machine Learning Crash Course, Large language models. Supports token-prediction and limitation explanations; the offline assistant uses clearly labelled authored responses.

S12. Google, Nearby Connections overview. Candidate transport for the later school service, subject to actual device and integration tests.

S13. Android documentation, App signing. Supports the distinction between a content package and a signed app update.

Technical references inform accuracy, not implementation status. The numerical economy, quest names, dataset sizes, effort ranges and milestone boundaries are Chainy design proposals. The visual pack's accompanying notes retain generation prompts and limitations for production handoff.